

# ¿Quién necesita C++?

Fortran al rescate para programar CPUs y GPUs de manera portable

---

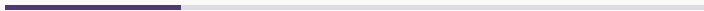
**Jorge Luis Gálvez Vallejo (NCI)**  
**jorge.galvezvallejo@anu.edu.au**

NCI  
Australian National University  
Canberra, Australia / 21 de agosto de 2026

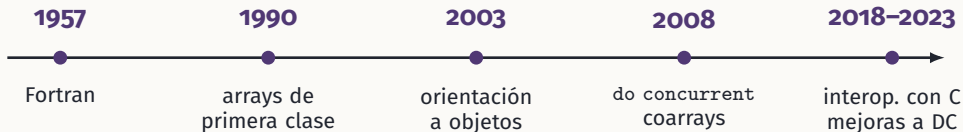


1

**Casi 70 años**



# El lenguaje de alto nivel más viejo del mundo



No es un lenguaje viejo. Es un lenguaje **con historia**.

Casi todo lo que la gente critica de Fortran se decidió **antes de 1990**.

# 1 La mala fama

## Problemas de verdad

- **Compiladores:** una pesadilla: GNU, Intel, NVIDIA, Cray, AMD, Flang
- **Compatibilidad:** el código de 1980 todavía compila — y el lenguaje carga con todo

## Y el problema de fondo

- Cada vez **menos usuarios**
- **La docencia se fue:** métodos numéricos y física computacional ahora se enseñan con Python, C++, Julia

*C y C++ se volvieron la alternativa para el cálculo científico moderno. Fortran perdió la clientela.*

# 1 Sin embargo

- Arrays de primera clase — rebanadas, elemental, reducciones, sin escribir el loop
- Orientación a objetos **F2003**
- Memoria dinámica (casi) segura — allocatable, move\_alloc
- Paralelismo nativo **en el estándar F2008**

## El meollo del asunto:

Fortran permite escribir código eficiente sin tener que haber estudiado ciencias de la computación.

**2**

**Un solo código**



## 2 El mismo loop donde sea!

### Fortran estándar

```
do concurrent (k=1:nk,j=1:nj,i=1:ni)
  c(i,j,k) = al*a(i,j,k) + b(i,j,k)
end do
```

### El muro de directivas

```
!$omp target teams loop
  collapse(3)
do k = 1, nk
  do j = 1, nj
    do i = 1, ni
      c(i,j,k) = alpha*a(i,j,k)
      + b(i,j,k)
    end do
  end do
end do
```

## 2

# Dónde sí y dónde no?

Cada vendor hace offload a *su* GPU

| Compiler          | DC → CPU | DC → GPU | Notes / flags                                                  |
|-------------------|----------|----------|----------------------------------------------------------------|
| nvfortran (NVHPC) | Yes      | Yes      | most mature; -stdpar=gpu; managed mem; F2023 reduce/locality   |
| ifx (Intel)       | Yes      | Yes      | DC lowered to OpenMP; -fopenmp-target-do-concurrent; ifort EOL |
| ftn (Cray CCE)    | Yes      | Yes      | AMD + NVIDIA, unified mem; -h thread_do_concurrent             |
| flang/aof (AOCC)  | Yes      | No       | x86/Zen only; -mp; → llvm-flang in 6.0                         |
| xlf (IBM XL)      | Yes      | Part.    | POWER+NVLink only; <i>legacy</i> (Open XL)                     |
| frt (Fujitsu)     | Yes      | No       | SVE auto-vec; -Kparallel; not DC-aware                         |
| gfortran (GCC)    | Part.    | No       | generic autopar -ftree-parallelize-loops=N; not DC-aware       |
| LLVM flang        | Part.    | Part.    | emerging; -fdo-concurrent-to-openmp=host device                |
| lfortran          | Part.    | Part.    | beta; unified mem; not production-ready                        |
| nagfor (NAG)      | No       | No       | serial; correctness oracle, not a perf path                    |



## 2 Coarrays

### La promesa

F2008. El diseño es **elegante**: memoria distribuida dentro del lenguaje, sin biblioteca externa.

### La realidad

GCC necesita OpenCoarrays. Intel trae el suyo. El de Cray es bueno. **NVIDIA y AMD prácticamente no existen.**

### Mi modelo mental no es: “nativo es bueno”

Es: **portable y listo para la vida real, o no cuenta.**

`do concurrent` lo pasa. Los coarrays *todavía* no.

Para memoria distribuida uso **MPI**; por eso existe `pic-mpi`. Podría añadirle un backend de coarrays — decidí que no: la mayoría de los runtimes de coarrays están implementados **sobre MPI**, así que estaría envolviendo una envoltura.

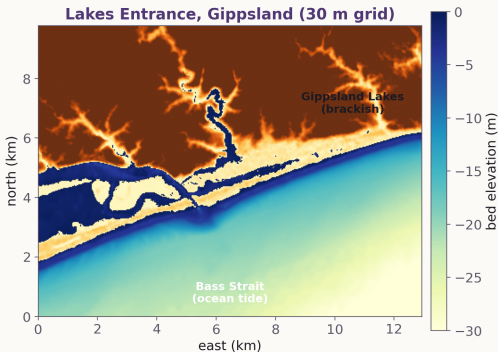
3

**Rakali**



### 3 Rakali: una historia de éxito

Simulación hidrodinámica — costas, ríos, inundaciones, océano abierto — 100 %  
Fortran



- Lakes Entrance, Gippsland
- Malla de **30 m**
- Marea del estrecho de Bass contra los lagos salobres

#### Portabilidad real

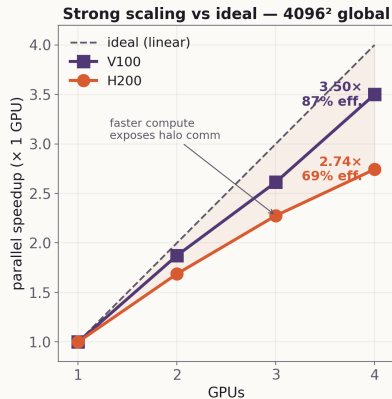
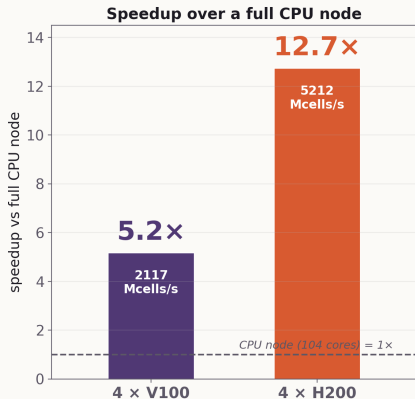
**GPUs:** NVIDIA, AMD, Intel

**CPUs:** Apple, ARM, Intel, AMD

Un solo código fuente.

## 3

## Los números



Contra un nodo CPU completo (104 núcleos): **5.2x** con 4xV100, **12.7x** con 4xH200.

### 3 La parte honesta

#### Mejor hardware, peor eficiencia

Escalamiento fuerte a 4 GPUs:

V100 — **87 %** de eficiencia

H200 — **69 %** de eficiencia

El cómputo más rápido **expone la comunicación de halos**.

#### Por qué lo cuento

Ante un público de ingeniería de software, el resultado que *no* salió como uno quería vale más que el que sí.

**4**

## **Lo que costó**

---



## 4 El lenguaje es portable. El ecosistema no.

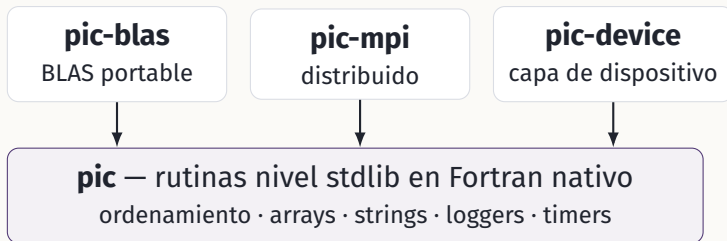
Rakali funciona. Pero **mantenerlo** reveló el problema de fondo:

- **stdlib** es muy buena — y en la práctica compila con GNU
- `mpi_f08` necesita **vapaa** (Hammond) para cruzar compiladores
- Cada proyecto reimplementa las mismas cinco cosas: ordenamiento, logging, timers, strings, pegamento para BLAS

Ese hueco es de **ingeniería**, no de ciencia.

Y por eso es nuestro trabajo.

## 4 El ecosistema pic



interoperabilidad, porque nunca estás atrapado: **libfint** (libcint, portado) · interfaz a **cuEST**

**fpm**

**CMake**

**compilar en todos lados!**



## 4 No es una biblioteca de conveniencia: es infraestructura

### metalquicha

Química cuántica. Tres motores (`tblite`, `libcint`, `cuEST`) tras **una sola interfaz**.

El camino CPU existe para **estar en desacuerdo** con el de GPU — y todo se valida contra PySCF.

### terco

Integrales de repulsión electrónica en GPU, **solo con** `do concurrent`.

Una **V100 de 2017**: **3.5**× más rápido que Q-Chem con 104 hilos en Sapphire Rapids.

Hidrodinámica y química cuántica sobre la misma base.

Eso es lo que convierte a `pic` en infraestructura.

*Si ignoramos la propaganda contra Fortran , hay un lenguaje muy poderoso al alcance de cualquiera.*

Pero no se sostiene solo. Necesita **CI, empaquetado, pruebas, diseño de APIs.**

C++ no tiene todas las respuestas. Tiene más **ingeniería.**

```
pic · pic-blas · pic-mpi · pic-device  
rakali · metalquicha · terco · libfint  
github.com/JorgeG94 · fortran-lang.org
```

## 4 Respaldo: MOM6, un tercer testigo

- **200 mil líneas** de código climático que *no* escribí
- Mismo enfoque: `do concurrent` para el cómputo, directivas solo para mover datos
- Requisito heredado y no negociable: **reproducibilidad bit a bit**

*Clima y química cuántica, códigos distintos, misma conclusión.*

# Gracias

No todo necesita Fortran, pero no tampoco todo necesita C++.

Jorge Luis Gálvez Vallejo (NCI)

jorge.galvezvallejo@anu.edu.au



[github.com/JorgeG94/pic](https://github.com/JorgeG94/pic)

